

Firefighting Drone Swarm – Iteration 1 Documentation

Course: SYSC 3303A – Real-Time Concurrent Systems (Winter 2026)

Team: Umniyah Mohammed, Armin Mozafari, Liam Perreault, Vincent Nguyen

Iteration: 1 – Basic Communication and Thread Infrastructure

1. Team Contributions

Team Member	Contributions
Vincent Nguyen	Implemented the Scheduler thread, designed the minimal GUI framework, created sequence diagrams, and wrote the README instructions.
Liam Perreault	Implemented the Drone and FireIncident subsystem threads.
Armin Mozafari	Developed UML diagrams and wrote unit/integration tests.
Umniah Mohammed	Implemented the shared infrastructure, including the MessageBus and synchronized MonitorQueue for thread-safe inter-subsystem communication, as well as a minimal integration main.

2. Iteration 1 Summary

Iteration 1 focused on establishing the **basic inter-thread communication** for the Firefighting Drone Swarm project. The main objectives were:

- Implement a **thread-safe infrastructure** (MessageBus and MonitorQueue) to allow decoupled communication between subsystems.
- Establish **Scheduler, Drone, and FireIncident threads**, simulating event-driven behavior for drone dispatching.
- Create a **minimal GUI framework** for monitoring purposes.
- Ensure that **FireEvents** could flow from the FireIncident subsystem to the Scheduler, be dispatched to a Drone, and generate DroneStatus updates back to the Scheduler and FireIncident subsystem.
- Validate the setup with **unit and integration tests**.

3. Subsystem Responsibilities

3.1 Scheduler

- Receives fire events from the FireIncident subsystem.
- Determines which drone to dispatch (single-drone assumption for this iteration).
- Sends commands to the Drone subsystem.
- Monitors drone status updates and marks drones as available after completion.

3.2 Drone Subsystem

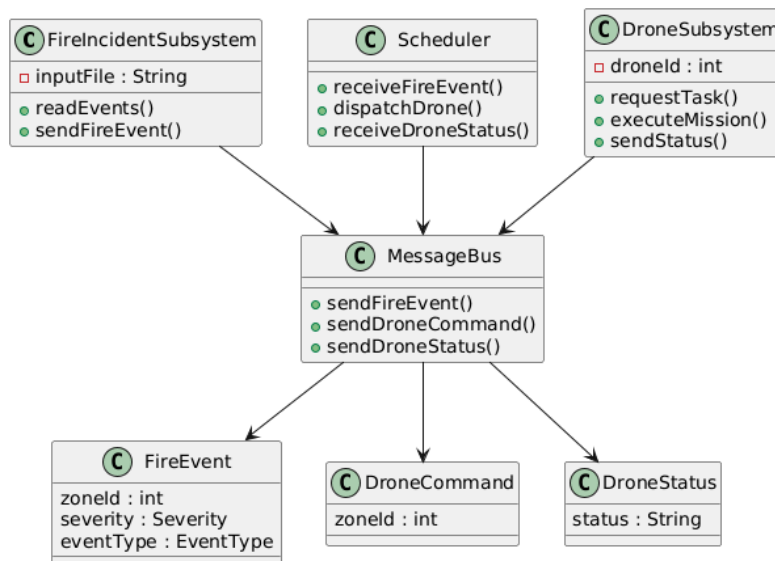
- Receives commands from the Scheduler.
- Simulates drone travel and agent deployment.
- Sends status updates back to the Scheduler.

3.3 FireIncident Subsystem

- Reads fire events from CSV input files.
- Sends events to the Scheduler.
- Monitors DroneStatus updates for confirmation of fire service.

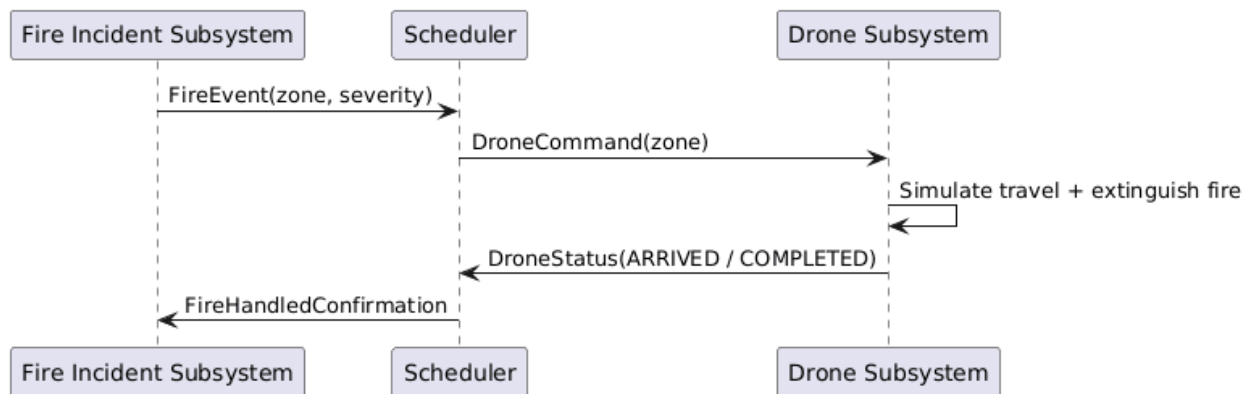
4. UML Class Diagram

Description: This UML diagram represents the main classes and their relationships in Iteration 1. It includes the subsystems, message structures, and shared infrastructure.



5. Sequence Diagram

Description: The sequence diagram illustrates the interactions between the FireIncident subsystem, Scheduler, and Drone subsystem for a single fire event in Iteration 1.



6. Design Notes

- **Thread-safe communication:** MonitorQueue uses synchronized, wait(), and notifyAll() to avoid busy-waiting.
- **Producer-consumer model:** FireIncident subsystem produces events, Scheduler consumes and dispatches, Drone produces statuses, Scheduler consumes statuses.
- **Minimal scheduling logic:** Iteration 1 uses a single drone and simple dispatch without queueing. Future iterations will expand this.
- **Integration:** A minimal main program validates end-to-end message flow between subsystems.

7. Iteration Highlights

- All three subsystems successfully communicate via MessageBus.
- Scheduler correctly dispatches drone and receives status updates.
- Infrastructure supports multiple threads and can scale to multiple drones in future iterations.
- Unit and integration tests validate communication and subsystem behaviors.

Next Steps

- Implement multi-drone scheduling and queue management (Iteration 2).
- Add UDP-based multi-process communication.
- Enhance GUI for real-time monitoring of drones and fire incidents.